

第5章

时钟系统与休眠模式



第5章 时钟系统与休眠模式

- 5.1 时钟系统
- 5.2 休眠模式

5.1 时钟系统

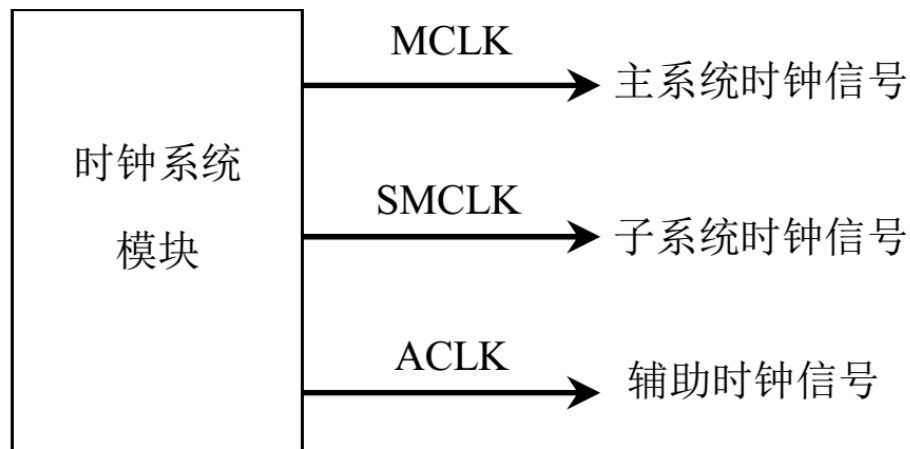


- 单片机之所以能够有序地工作，是因为整个单片机在统一时钟信号的作用下，由CPU协调各个功能模块进行各种操作。
- 这里的时钟信号是单片机运行的基准信号，各个功能模块所需的时钟也都是以此为基准，进行分频得到的。
- 产生时钟信号的模块一般称为时钟模块。他是单片机的动力之源。
- MSP430种类繁多，各个系列之间时钟系统的内部构成均不尽相同。

5.1 时钟系统



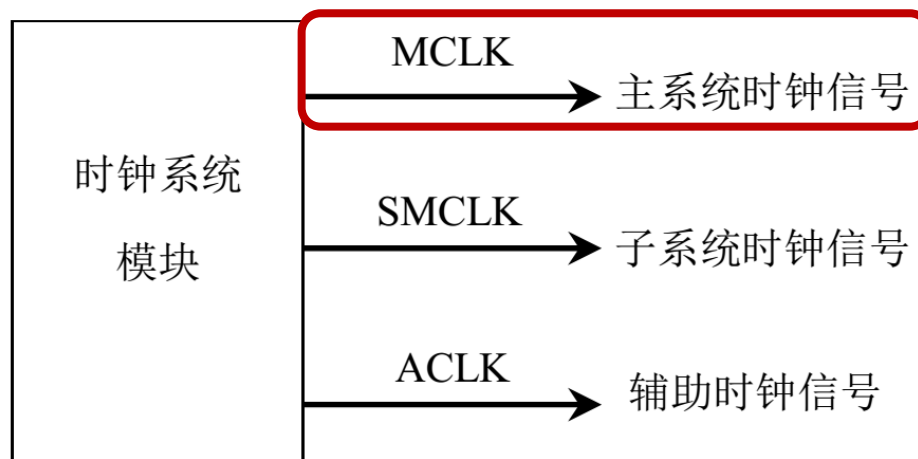
- 每个时钟模块均有三路不同的输出，如图所示，分别是主系统时钟、子系统时钟以及辅助时钟。



5.1 时钟系统



● 主系统时钟



- 主要是为CPU提供运行时钟，但也可用于其他高速模块(如定时器和数模转换模块)。
- 一般地，MCLK越高，运行速度就越快。
- 为了使CPU的性能能够充分发挥，一般使CPU的工作频率不低于1MHz。

5.1 时钟系统

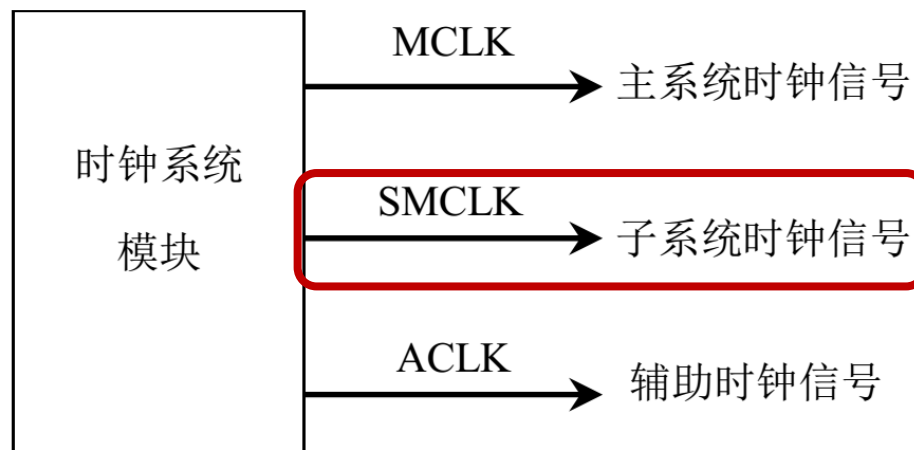


- 需要注意的是，当MCLK关闭时，CPU也将随之关闭即中止工作。这一特点在低功耗应用设计中广泛使用。
- 例如，由于CPU是单片机中耗电量较大的部件。由于大多数应用中CPU并不需要一直工作，因此，可使仅在需要的时候打开MCLK，让CPU工作而其他时间让CPU不工作从而达到降低功耗的目的。

5.1 时钟系统



● 子系统时钟

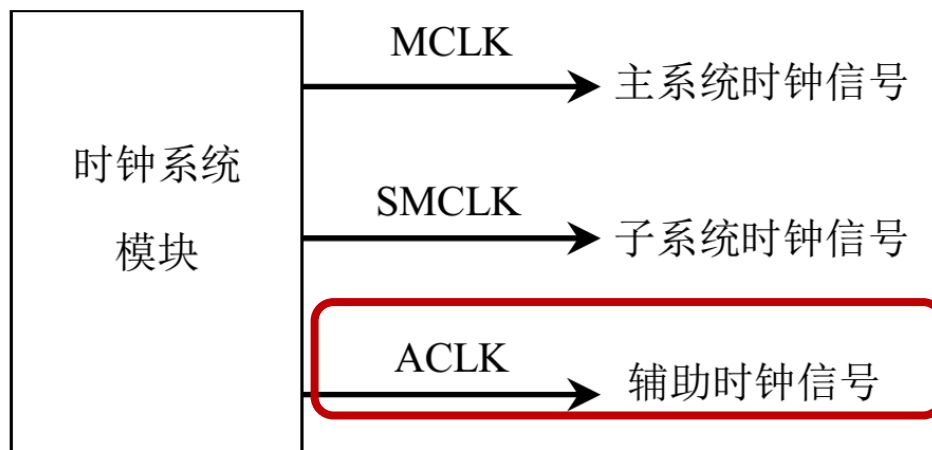


- 主要为片内一些高速外围设备提供高速时钟。
- SMCLK与MCLK是相互独立的，即关闭MCLK不会影响SMCLK。也就是在关闭CPU的情况下，使用SMCLK的片内外围设备仍可以继续工作。
- 这一特点也广泛应用在低功耗应用设计中。

5.1 时钟系统



● 辅助时钟



- 主要为片内一些低速外围设备提供低速时钟。
- ACLK与SMCLK、MCLK之间也是彼此独立的。
- 在单片机运行期间，该时钟一般不关闭。其作用是产生用于唤醒CPU的基本时钟信号，如可与定时器配合定时唤醒CPU。

5.1 时钟系统



- 5.1.1 时钟系统结构

- 5.1.2 时钟失效处理

5.1.1 时钟系统结构

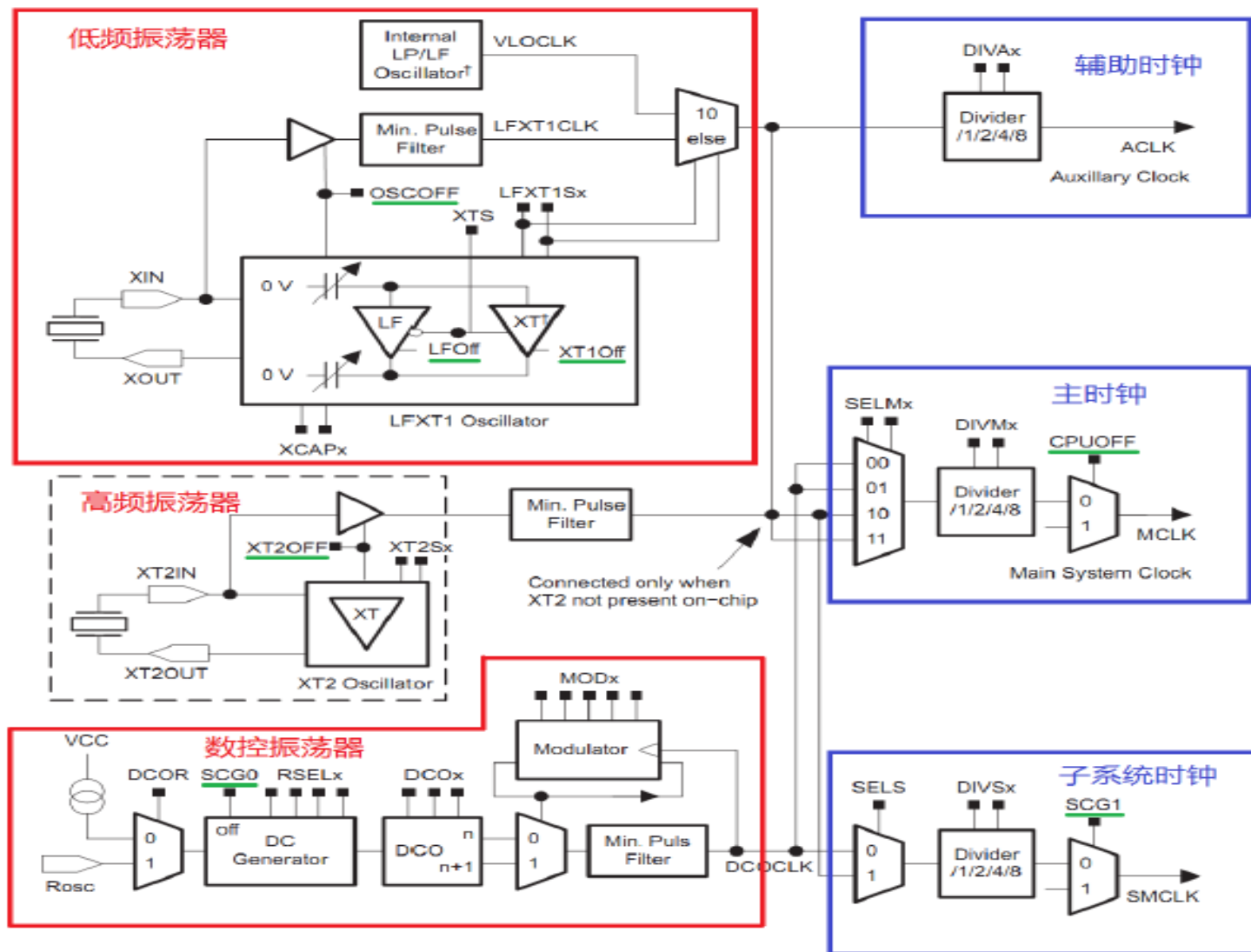
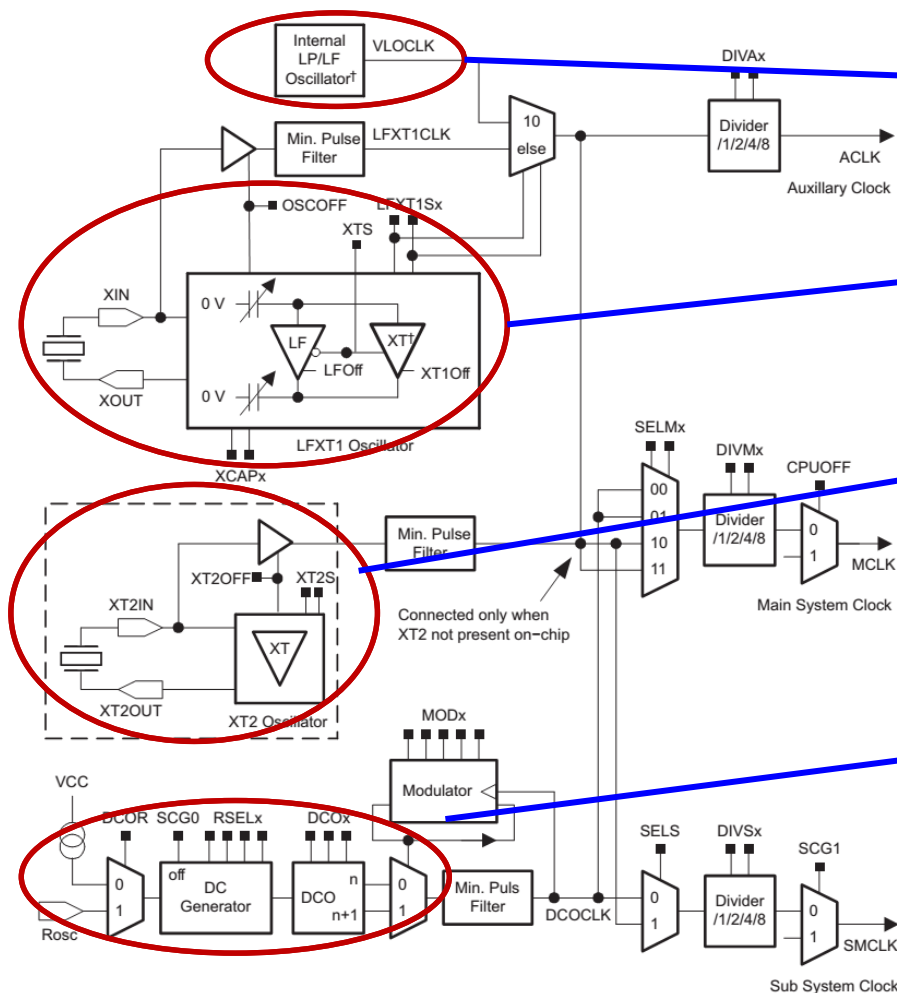


图 4.1 Basic Clock Module+ 功能框图

5.1.1 时钟系统结构



1.时钟源



内部低速低功耗时钟振荡器(VLO)

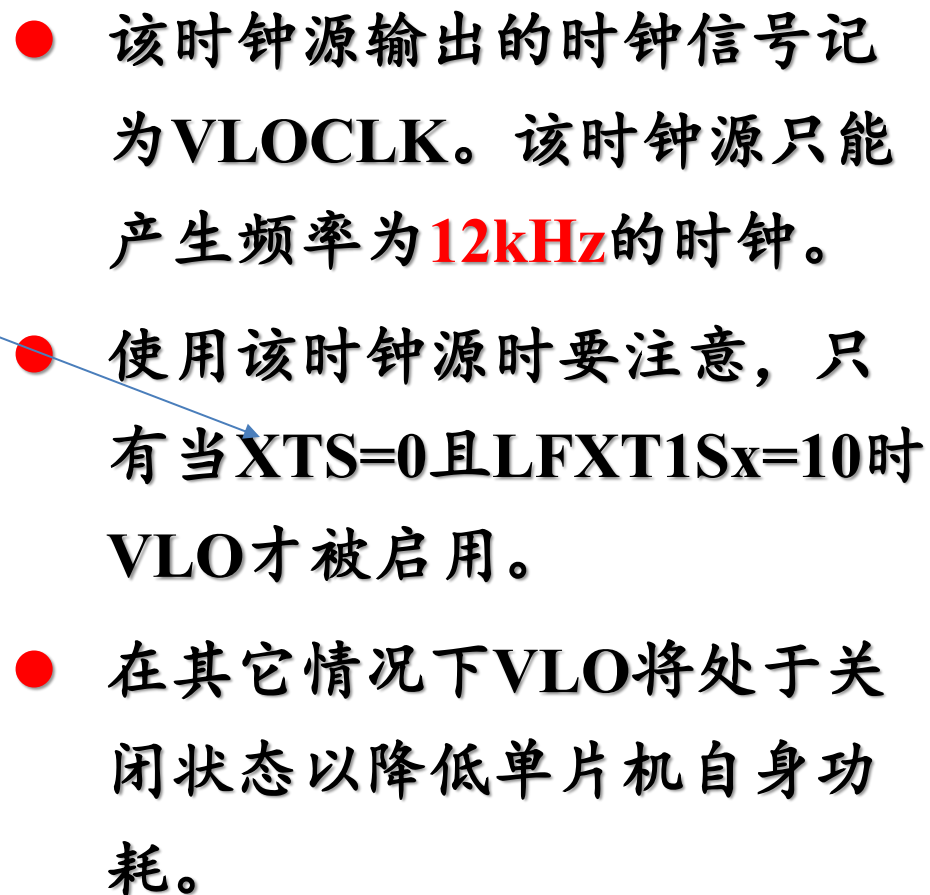
外部低速时钟振荡器(LFXT1)

外部高速时钟振荡器(XT2)
振荡器（目前MSP430G系列单片机不支持）。

内部数控时钟振荡器(DCO,
实际是一个数字可控的RC振荡器)

MSP430G系列单片机目前只能通过内部数控振荡器DCO来获得高频时钟

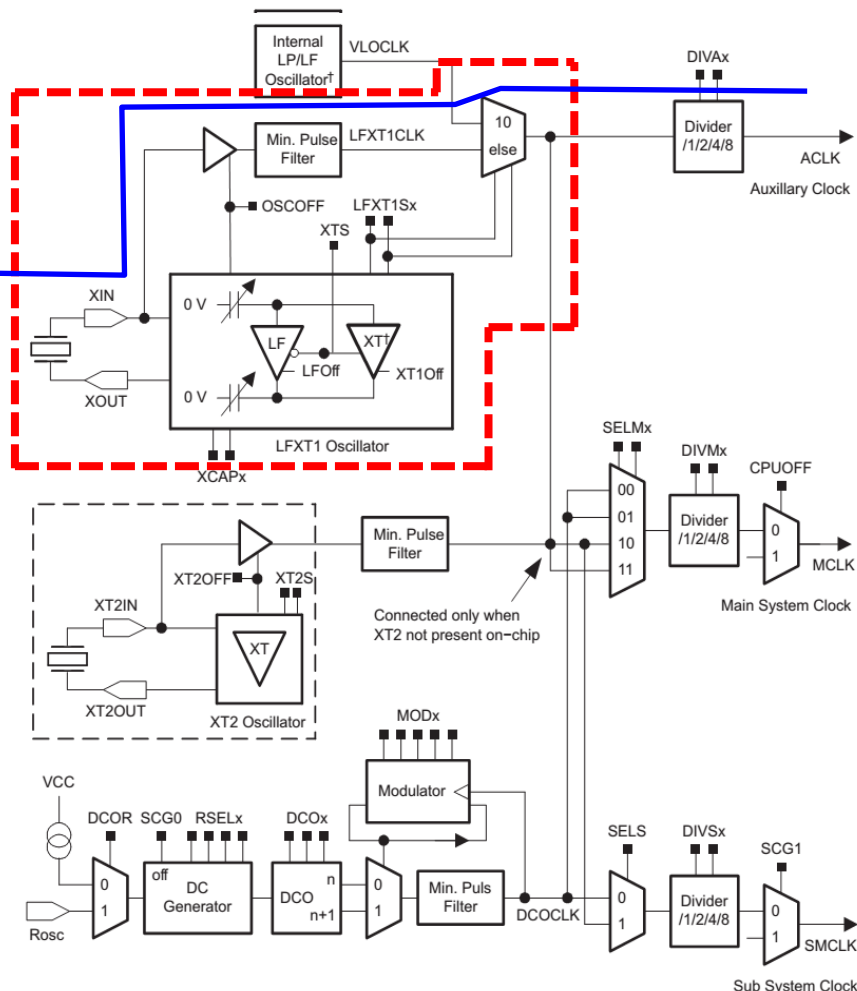
1) 内部低速时钟振荡器(VLO)



5.1.1 时钟系统结构



1. 时钟源



(2) 外部低速时钟振荡器(LFXT1)

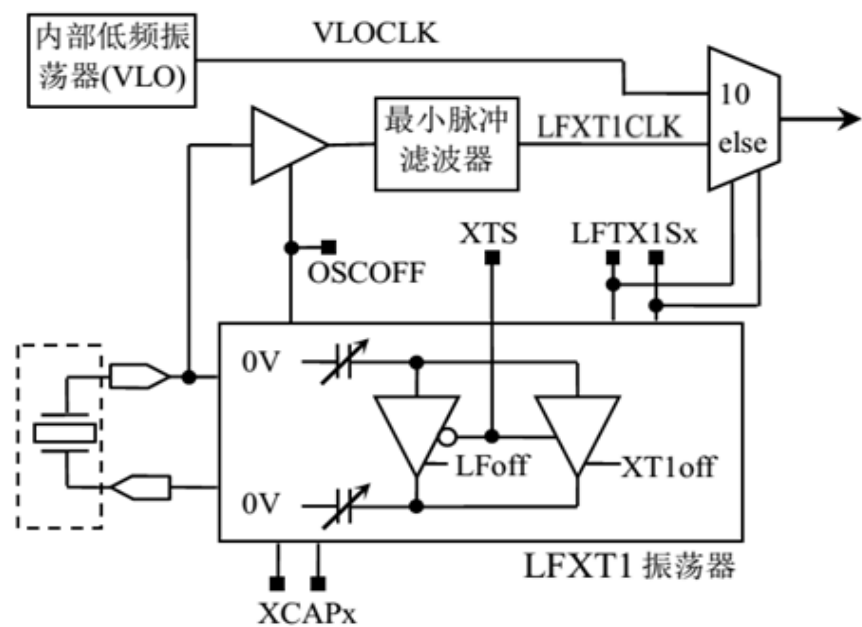
- 它有两种工作方式，一种是直接外部时钟信号，并以此信号作为时钟源的输出信号(LFXT1CLK);
- 若使用该方式需要使 $OSCOFF=0$ 、 $LFXT1Sx=11$ 、 $XCAPx=00$ 。这时外部时钟信号的频率与LFXT1CLK的频率一致。



5.1.1 时钟系统结构

(2)外部低速时钟振荡器(LFXT1)

- 另一种是与外接晶振组成时钟振荡器（LFXT1），振荡器的输出即为LFXT1CLK。
- LFXT1既可以产生高频时钟信号，也可以产生低频时钟信号。具体工作方式由控制位XTS和LFXT1Sx共同决定。（LFoff和XT1off除受控于XTS外，还受其他信号的控制；XCAPx用于配制低频模式下内部匹配电容的值）

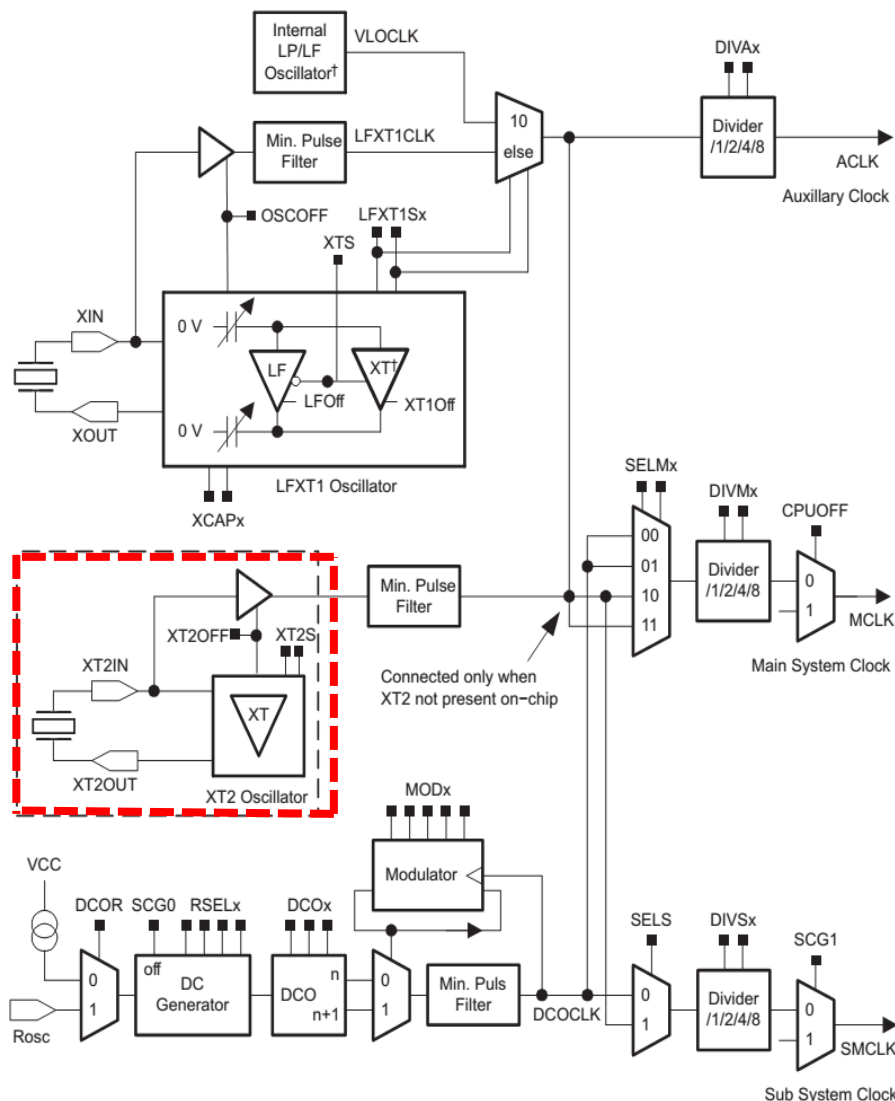


VLOCLK 与 LFXT1CLK 结构示意图

LFXT1 的工作方式

XTS	LFXT1Sx	功能说明
0	00	表示 32.768KHz 晶振接在 LFXT1 上
0	01	保留
0	10	选择 VLOCLK
0	11	使用外部时钟源
1	00	使用 0.4-1MHz 的晶振
1	01	使用 1-3MHz 的晶振
1	10	使用 3-16MHz 的晶振
1	11	使用外部时钟源(0.4-16MHz)

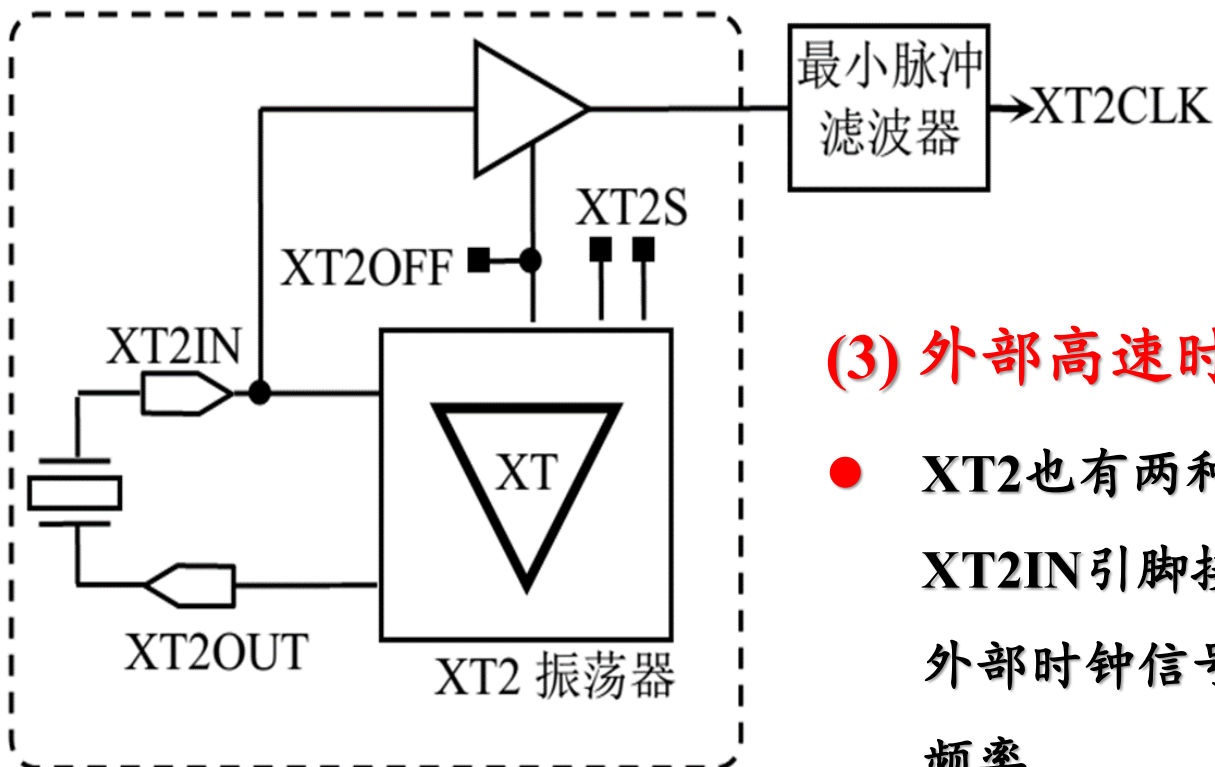
5.1.1 时钟系统结构



(3) 外部高速时钟振荡器(XT2)

- 除了LFXT1外，还具有另外一个外部时钟源XT2，其输出的时钟信号为XT2CLK。

5.1.1 时钟系统结构

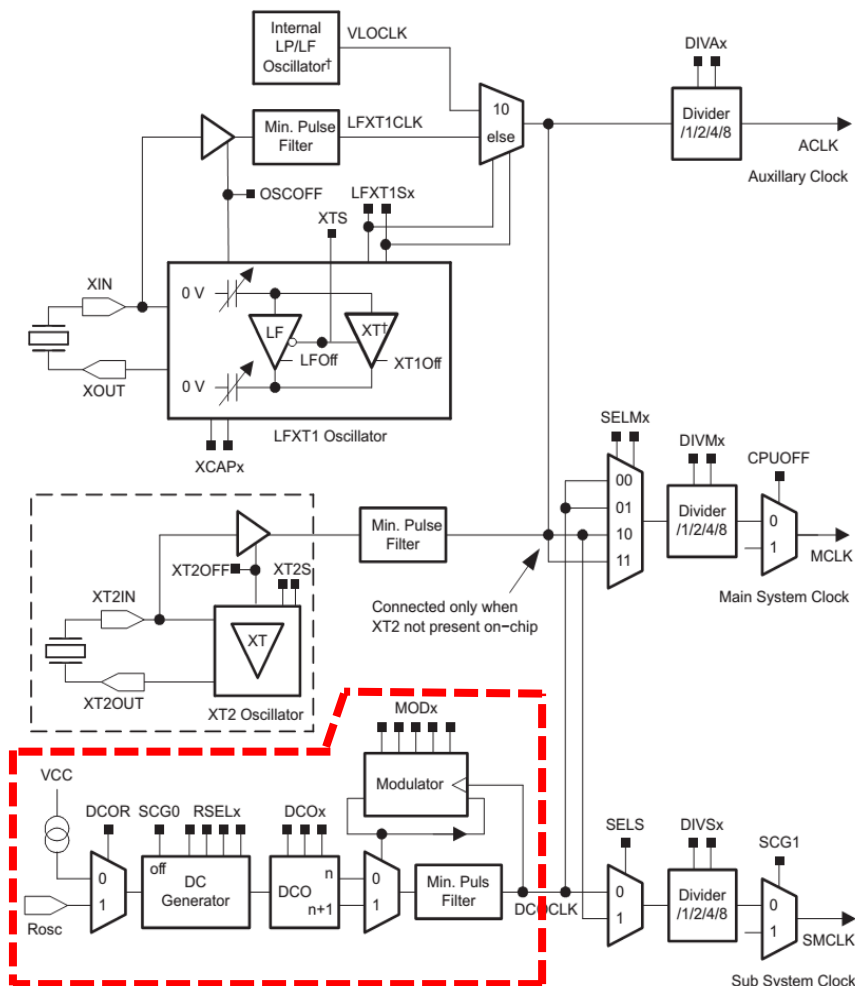


XT2 逻辑结构图

(3) 外部高速时钟振荡器(XT2)

- XT2也有两种工作方式，一种是通过XT2IN引脚接入外部时钟信号，此时外部时钟信号的频率即是XT2CLK的频率。
- 另外一种工作方式是与外接高频晶振构成振荡器，其工作方式与处于高频模式的LFXT1一样。

5.1.1 时钟系统结构



(4) 数控时钟振荡器(DCO)

- 主要包括直流发生器、数控振荡器、调整器和最小脉冲滤波器等部分。
- 由于该DCO属于流控振荡器，所以直流发生器的作用是为DCO提供工作所需的电流。
- 调整器的作用主要是对DCO输出的时钟进行更为精细的调整，使时钟更加精准。
- 最小脉冲滤波器(Minimum Pulse Clock Filter)的作用是防止输出的时钟中混有其它频率成分。因此，该滤波器可以提高时钟的可靠性和准确性。

5.1.1 时钟系统结构



(4) 数控时钟振荡器(DCO)

- **DCOR**表示是否使能外部电阻（由于DCO的振荡频率取决于通过电阻流入直流发生器的电流大小，因此除使用MSP430的内部电阻之外，用户可以选择外接的电阻。）
- **SCG0**为直流发生器开关
- **RSELx**、**DCOx**与**MODx**共同决定输出频率

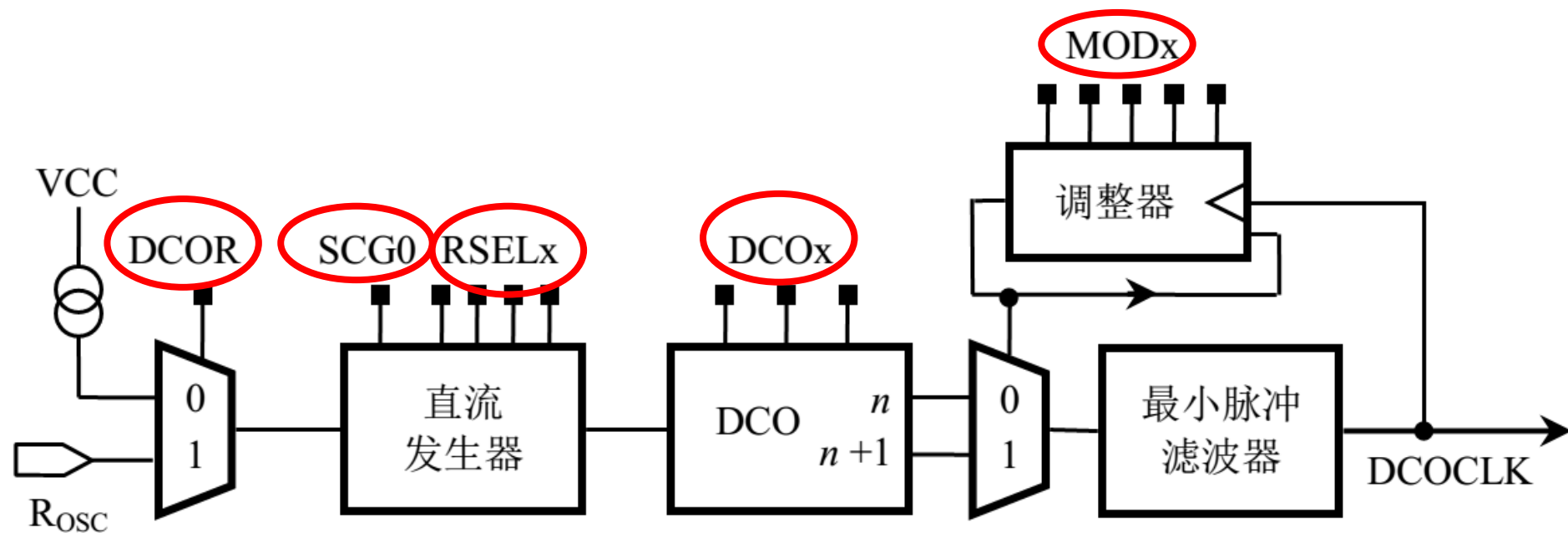


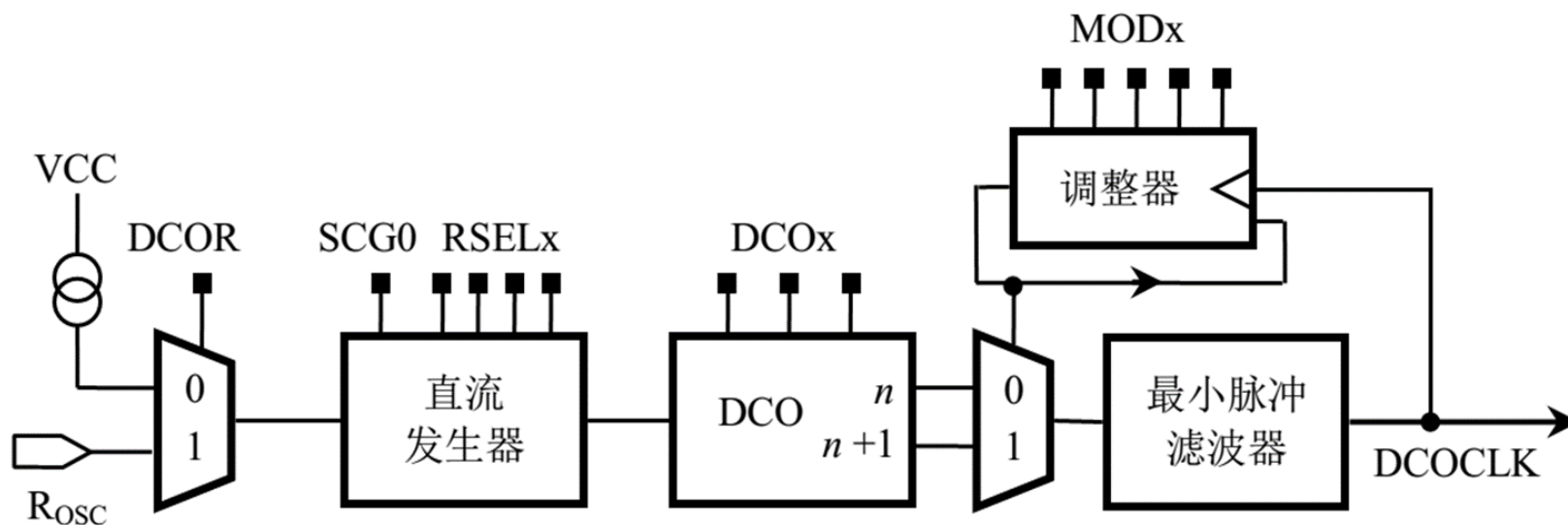
图 6.5 DCO 逻辑结构图

5.1.1 时钟系统结构



保证频率精度，频率设置分3步：

- 设置RSEL_x确定时钟的标称频率；
- 选择DCO_x，在标称频率基础上分段粗调；
- 选择MOD_x，在粗调的基础上再细调。



DCO 逻辑结构图

5.1.1 时钟系统结构



RSEL_x（位于控制寄存器BCSCTL1的低4位）：16个标称频率段。

DCO_x（位于控制寄存器DCOCTL的高3位）：一个标称频率粗略分为8个区域。

MOD_x（位于控制寄存器DCOCTL的低5位）：每个区域再划分为32个微调区间。

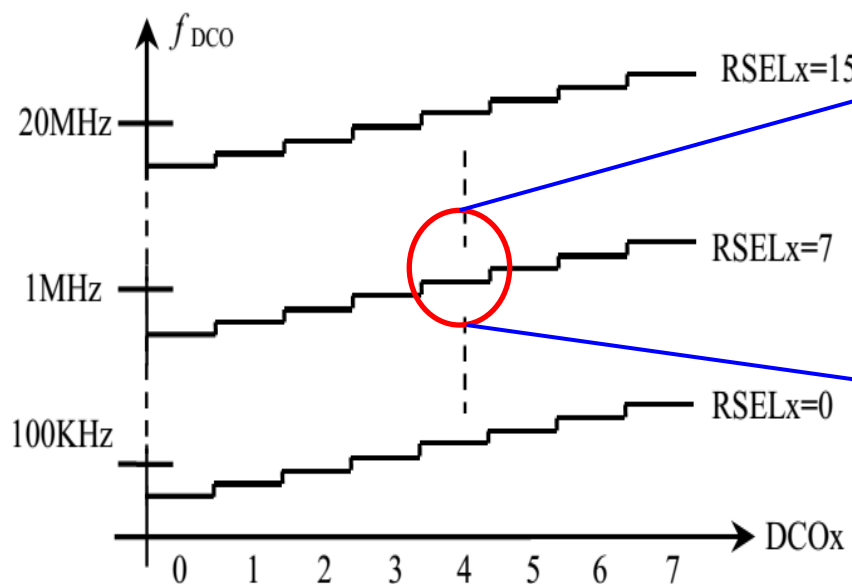


图 6.7 $RSEL_x$ 、 DCO_x 与 f_{DCO} 的关系

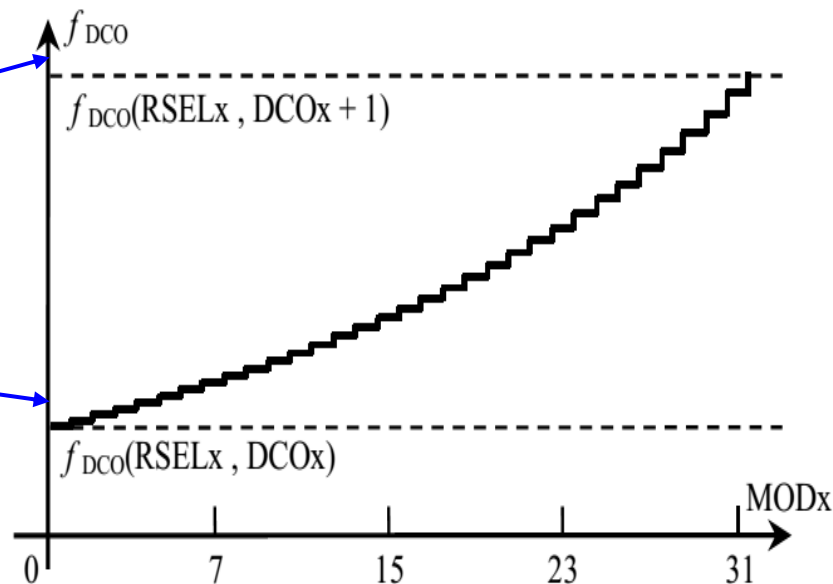
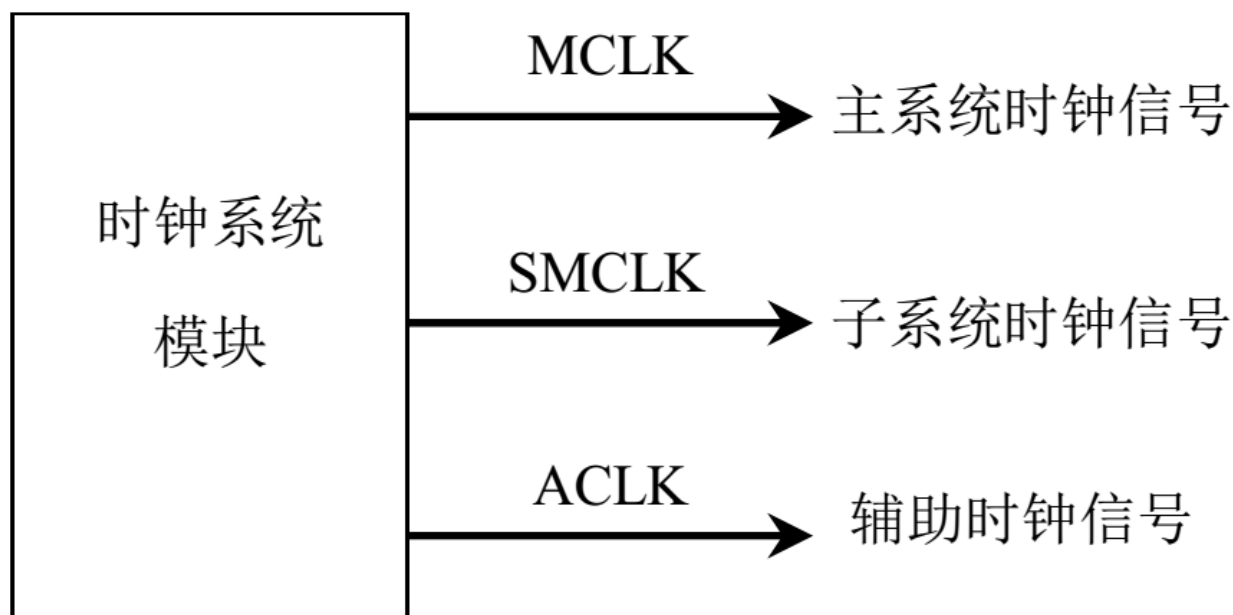


图 6.8 f_{DCO} 与 MOD_x 的关系



5.1.1 时钟系统结构

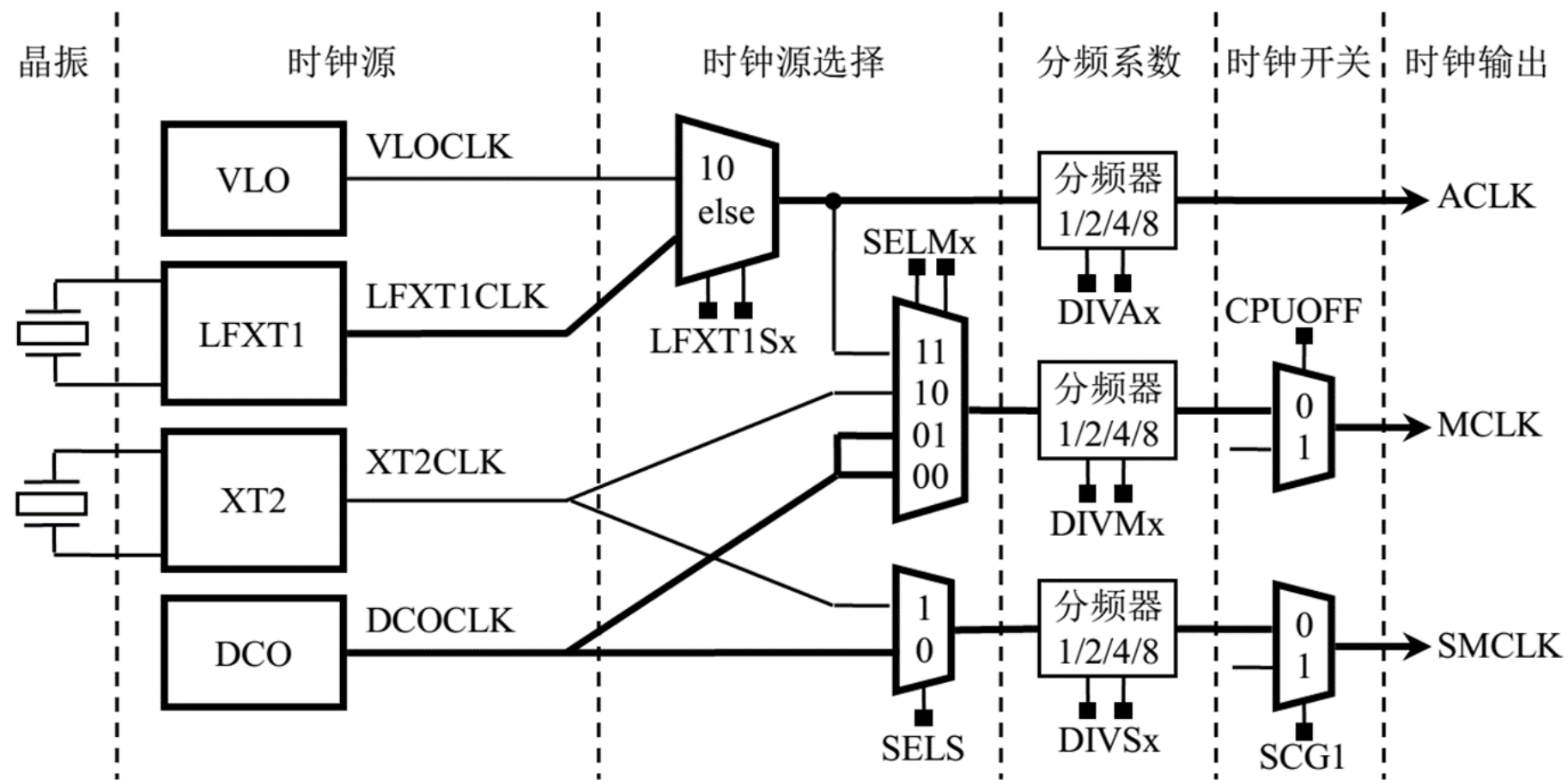
2.输出时钟信号





5.1.1 时钟系统结构

2.输出时钟信号



时钟源与输出时钟的对应关系



5.1.1 时钟系统结构

2.输出时钟信号

● 时钟系统寄存器

The basic clock module+ registers are listed in [Table 5-1](#).

Table 5-1. Basic Clock Module+ Registers

Register	Short Form	Register Type	Address	Initial State
DCO control register	DCOCTL	Read/write	056h	060h with PUC
Basic clock system control 1	BCSCTL1	Read/write	057h	087h with POR ⁽¹⁾
Basic clock system control 2	BCSCTL2	Read/write	058h	Reset with PUC
Basic clock system control 3	BCSCTL3	Read/write	053h	005h with PUC ⁽²⁾
SFR interrupt enable register 1	IE1	Read/write	000h	Reset with PUC
SFR interrupt flag register 1	IFG1	Read/write	002h	Reset with PUC

⁽¹⁾ Some of the register bits are also PUC initialized (see [Section 5.3.2](#)).

⁽²⁾ The initial state of BCSCTL3 is 000h in the MSP430AFE2xx devices.



5.1.1 时钟系统结构

2.输出时钟信号

● 时钟系统寄存器

	7	6	5	4	3	2	1	0
BCSCTL1	XT2OFF	XTS	DIVA _x		RSEL _x			
	rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

	7	6	5	4	3	2	1	0
BCSCTL2	SELM _x		DIVM _x		SELS	DIVS _x		DCOR
	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

	7	6	5	4	3	2	1	0
BCSCTL3	XT2S _x		LFXT1S _x		XCAP _x		XT2OF	LFXT1OF
	rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r0	r-(1)

	7	6	5	4	3	2	1	0
DCOCTL	DCO _x			MOD _x				
	rw-0	rw-1	rw-1	rw-0	rw-0	rw-0	rw-0	rw-0

rw-0的意思是该位可读可写，复位后初始值为0



5.1.1 时钟系统结构

● 基本时钟系统控制寄存器1(BCSCTL1)

7	6	5	4	3	2	1	0
XT2OFF	XTS	DIVAx		RSELx			
rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

XT2OFF: 用于控制XT2时钟源的开关。

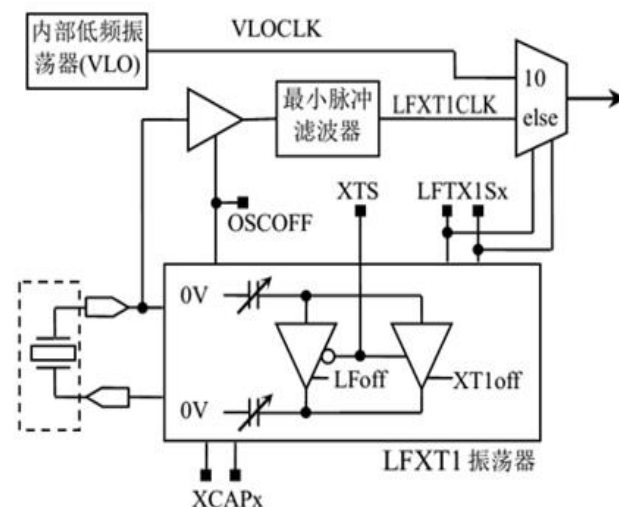
XT2OFF=1, XT2关闭;

XT2OFF=0, XT2打开。

XTS: 用于控制LFXT1时钟源的模式选择。

XTS=1, LFXT1工作在高频模式;

XTS=0, LFXT1工作在低频模式。



VLOCLK 与 LFXT1CLK 结构示意图



5.1.1 时钟系统结构

7	6	5	4	3	2	1	0
XT2OFF	XTS	DIVAx		RSELx			
rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

为记忆和使用方便，将每个控制位的名字宏定义一个常量，该常量的大小由该控制位的位置决定。

261x.h 头文件中的宏定义

```
#define XTS          (0x40) /* LFXCLK 0:Low Freq. / 1: High Freq. */  
#define XT2OFF      (0x80) /* Enable XT2CLK */
```

这样做的好处是在对应控制位赋值和清零时比较方便，不需要再关心该位的位置了。



5.1.1 时钟系统结构

例如：

```
BCSCTL1 |= XT2OFF;      //对控制位XT2OFF置1
BCSCTL1 &= ~XT2OFF;     //对控制位XT2OFF置0
```

```
BCSCTL1 |= 0x80;        //对控制位XT2OFF置1
BCSCTL1 &= ~0x80;       //对控制位XT2OFF置0
```

```
BCSCTL1 |= BIT7;        //对控制位XT2OFF置1
BCSCTL1 &= ~BIT7;       //对控制位XT2OFF置0
```

```
BCSCTL1 |= (1<<7);      //对控制位XT2OFF置1
BCSCTL1 &= ~(1<<7);     //对控制位XT2OFF置0
```



5.1.1 时钟系统结构

● 基本时钟控制寄存器1(BCSCTL1)

7	6	5	4	3	2	1	0
XT2OFF	XTS	DIVAx		RSELx			
rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

261x.h 头文件中的宏定义

```
#define DIVA0      (0x10) /* ACLK Divider 0 */
#define DIVA1      (0x20) /* ACLK Divider 1 */
#define DIVA_0     (0x00) /* ACLK Divider 0: /1 */
#define DIVA_1     (0x10) /* ACLK Divider 1: /2 */
#define DIVA_2     (0x20) /* ACLK Divider 2: /4 */
#define DIVA_3     (0x30) /* ACLK Divider 3: /8 */
```



5.1.1 时钟系统结构

例如：设置ACLK进行8分频

BCSCTL1 |= DIVA_3;

或

BCSCTL1 |= DIVA1+DIVA0;

注意：MSP430开机或复位后，大多数的控制位都会自动清零。因此，对寄存器首次设置时也都以此做为默认状态。

假如目前是2分频，想改为4分频，这样实现可以吗？

BCSCTL1 |= DIVA_2;

之前要加下面的语句

BCSCTL1 &= ~DIVA_1;



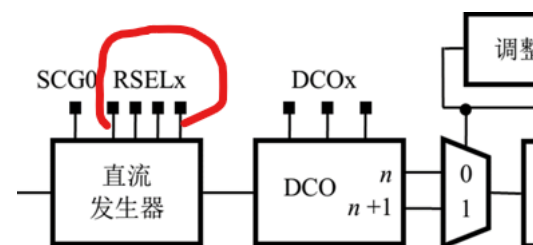
5.1.1 时钟系统结构

● 基本时钟控制寄存器1(BCSCTL1)

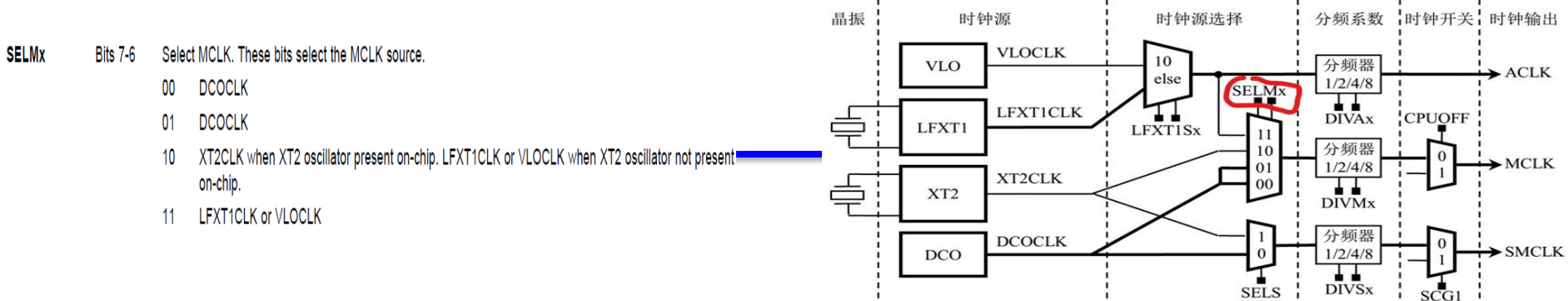
7	6	5	4	3	2	1	0
XT2OFF	XTS	DIVAx		RSELx			
rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

261x.h 头文件中的宏定义

```
#define RSEL0      (0x01) /* Range Select Bit 0 */
#define RSEL1      (0x02) /* Range Select Bit 1 */
#define RSEL2      (0x04) /* Range Select Bit 2 */
#define RSEL3      (0x08) /* Range Select Bit 3 */
```



RSELx Bits 3-0 Range select. Sixteen different frequency ranges are available. The lowest frequency range is selected by setting RSELx = 0. RSEL3 is ignored when DCOR = 1.



基本时钟控制寄存器2(BCSCTL2)

7	6	5	4	3	2	1	0
SELMx		DIVMx		SELS	DIVSx		DCOR
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

261x.h 头文件中的宏定义

```

#define SELM0      (0x40) /* MCLK Source Select 0 */
#define SELM1      (0x80) /* MCLK Source Select 1 */
#define SELM_0     (0x00) /* MCLK Source Select 0: DCOCLK */
#define SELM_1     (0x40) /* MCLK Source Select 1: DCOCLK */
#define SELM_2     (0x80) /* MCLK Source Select 2: XT2CLK/LFXTCLK */
#define SELM_3     (0xC0) /* MCLK Source Select 3: LFXTCLK */

```



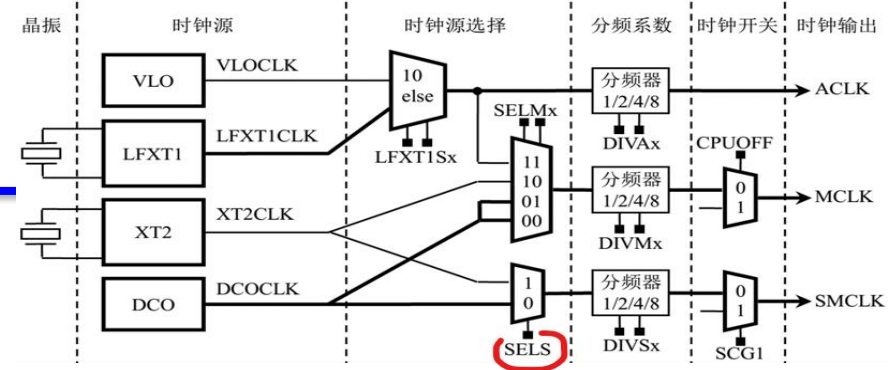
5.1.1 时钟系统结构

● 基本时钟控制寄存器2(BCSCTL2)

7	6	5	4	3	2	1	0
SELMx		DIVMx		SELS	DIVSx		DCOR
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

261x.h 头文件中的宏定义

```
#define DIVM0      (0x10) /* MCLK Divider 0 */
#define DIVM1      (0x20) /* MCLK Divider 1 */
#define DIVM_0     (0x00) /* MCLK Divider 0: /1 */
#define DIVM_1     (0x10) /* MCLK Divider 1: /2 */
#define DIVM_2     (0x20) /* MCLK Divider 2: /4 */
#define DIVM_3     (0x30) /* MCLK Divider 3: /8 */
```

● 基本时钟控制寄存器2(BCSCTL2)

7	6	5	4	3	2	1	0
SELMx		DIVMx		SELS	DIVSx		DCOR
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

261x.h 头文件中的宏定义

```
#define DIVS0          (0x02) /* SMCLK Divider 0 */
```

261x.h 头文件中的宏定义

```
#define DCOR           (0x01) /* Enable External Resistor : 1 */
```

```
#define SELS           (0x08) /* SMCLK Source Select 0:DCOCLK / 1:XT2CLK*/
```

```
#define DIVS_2         (0x04) /* SMCLK Divider 2: /4 */
```

```
#define DIVS_3         (0x06) /* SMCLK Divider 3: /8 */
```



5.1.1 时钟系统结构

● 基本时钟控制寄存器3(BCSCTL3)

7	6	5	4	3	2	1	0
XT2Sx		LFXT1Sx		XCAPx		XT2OF	LFXT1OF
rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r-0	r-(0)

261x.h : 261x.h 头文件中的宏定义

```
#define LFXT1S0      (0x10) /* Mode 0 for LFXT1 (XTS = 0) */
#define LFXT1S1      (0x20) /* Mode 1 for LFXT1 (XTS = 0) */
#define LFXT1S_0      (0x00) /* Mode 0 for LFXT1 : Normal operation */
#define LFXT1S_1      (0x10) /* Mode 1 for LFXT1 : Reserved */
#define LFXT1S_2      (0x20) /* Mode 2 for LFXT1 : VLO */
#define LFXT1S_3      (0x30) /* Mode 3 for LFXT1 : Digital input signal */
```



5.1.1 时钟系统结构

XTS	LFXT1Sx	功能说明
0	00	表示32.768KHz晶振接在LFXT1上
0	01	保留
0	10	选择VLOCLK
0	11	使用外部时钟源
1	00	使用0.4-1MHz的晶振
1	01	使用1-3MHz的晶振
1	10	使用3-16MHz的晶振
1	11	使用外部时钟源(0.4-16MHz)



5.1.1 时钟系统结构

● 基本时钟控制寄存器3(BCSCTL3)

7	6	5	4	3	2	1	0
XT2Sx		LFXT1Sx		XCAPx		XT2OF	LFXT1OF
rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r-0	r-(0)

261x.h 头文件中的宏定义

```
#define XCAP0      (0x04) /* XIN/XOUT Cap 0 */
#define XCAP1      (0x08) /* XIN/XOUT Cap 1 */
#define XCAP_0     (0x00) /* XIN/XOUT Cap : 0 pF */
#define XCAP_1     (0x04) /* XIN/XOUT Cap : 6 pF */
#define XCAP_2     (0x08) /* XIN/XOUT Cap : 10 pF */
#define XCAP_3     (0x0C) /* XIN/XOUT Cap : 12.5 pF */
```



5.1.1 时钟系统结构

● 基本时钟控制寄存器3(BCSCTL3)

7	6	5	4	3	2	1	0
XT2Sx		LFXT1Sx		XCAPx		XT2OF	LFXT1OF
rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r-0	r-(0)

XT2OF: 用于控制XT2振荡器的失效条件。

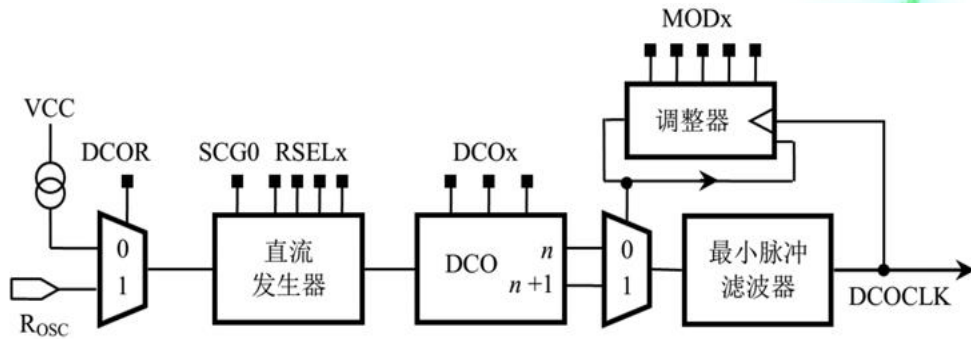
XT2OF=0 不存在失效条件

261x.h 头文件中的宏定义

```
#define LFXT1OF      (0x01) /* Low/high Frequency Oscillator Fault Flag */
```

```
#define XT2OF        (0x02) /* High frequency oscillator 2 fault flag */
```

LFXT1OF=1: 存在失效条件。



● 数控振荡控制寄存器 (DCOCTL)

7	6	5	4	3	2	1	0
DCOx			MODx				
rw-0	rw-1	rw-1	rw-0	rw-0	rw-0	rw-0	rw-0

该寄存器由DCOx和MODx控制，
8种频率。MODx居低位。

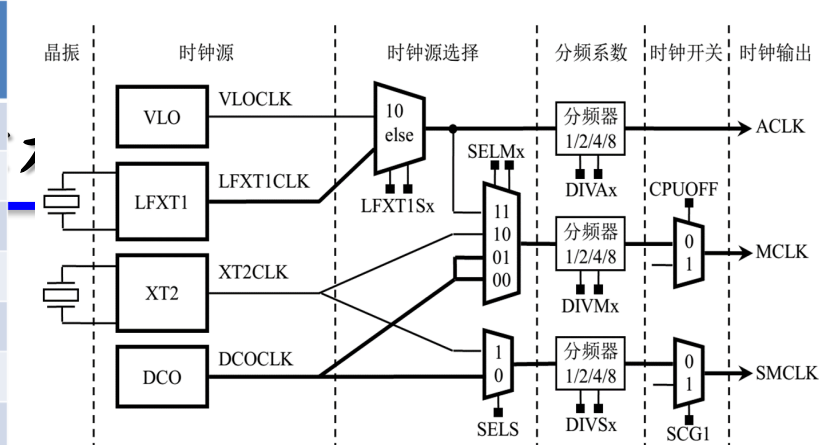
261x.h 头文件中的宏定义

```

#define MOD0      (0x01) /* Modulation Bit 0 */
#define MOD1      (0x02) /* Modulation Bit 1 */
#define MOD2      (0x04) /* Modulation Bit 2 */
#define MOD3      (0x08) /* Modulation Bit 3 */
#define MOD4      (0x10) /* Modulation Bit 4 */
#define DCO0      (0x20) /* DCO Select Bit 0 */
#define DCO1      (0x40) /* DCO Select Bit 1 */
#define DCO2      (0x80) /* DCO Select Bit 2 */

```

XTS	LFXT1Sx	功能说明
0	00	表示32.768KHz晶振接在LFXT1上
0	01	保留
0	10	选择VLOCLK
0	11	使用外部时钟源
1	00	使用0.4-1MHz的晶振
1	01	使用1-3MHz的晶振
1	10	使用3-16MHz的晶振
1	11	使用外部时钟源(0.4-16MHz)



时钟源与输出时钟的对应关系

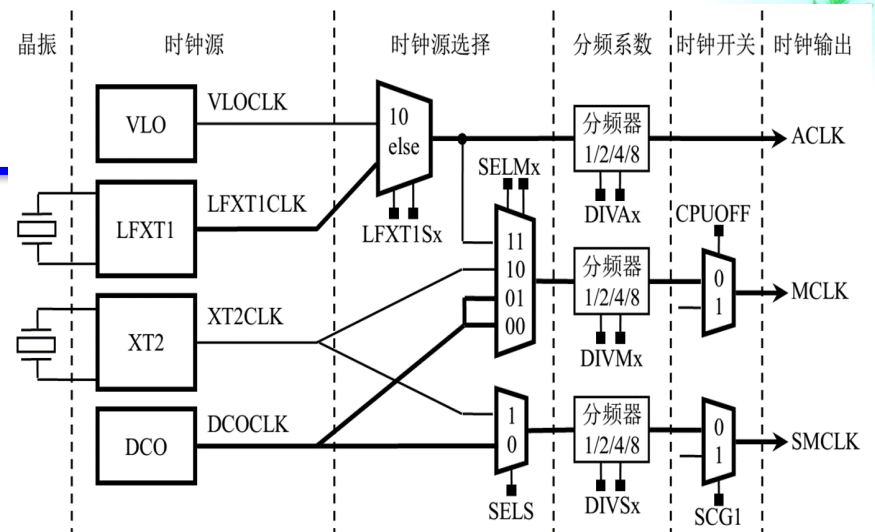
练习1：若XT1IN和XT1OUT外接32.768kHz的手表晶振，配制ACLK使其输出约4kHz时钟信号。

BCSCTL3 |= LFXT1S_0; //选中32.768
BCSCTL1 |= DIVA_3; //8分频

***说明：32.768kHz属于低频时钟，故XTS=0。**

练习2：若XT1IN处有一稳定的1MHz时钟信号，利用该信号做为ACLK的时钟，并使之输出250kHz的时钟。

BCSCTL3 |= LFXT1S_3; //使用外部高频晶振
BCSCTL1 |= DIVA_2+XTS; //4分频，且XTS为1表示高频时钟



时钟源与输出时钟的对应关系

练习3：若DCO为4MHz，试使SMCLK=DCOCLK，并使SMCLK输出500kHz的时钟。

BCSCTL2 |= DIVS_3; //选DCOCLK, 故SELS=0

练习4：试将MCLK和SMCLK分别配置为6、3MHz，其中DCO为默认12MHz的时钟。

BCSCTL2 |= DIVM_1 + DIVS_2; //MCLK=DCO/2, SMCLK=DCO/4

5.1.1 时钟系统结构



三种时钟的功能区别：

- 1) **MCLK: 主时钟 (Main system Clock)**，专为CPU运行提供的时钟。MCLK频率配置的越高，CPU执行的速度越快。虽然CPU速度越快功耗也越高，但高频率的MCLK可以让CPU工作时间更短。所以正确的低功耗设计并不是要尽量降低MCLK，而是在不用CPU时立刻关闭MCLK。大部分应用中，需要CPU运算的时间都非常短，所以，间歇开启MCLK（唤醒CPU）的方法节能效果非常明显。
- 2) **SMCLK: 子系统时钟 (Sub-main Clock)**，专为一些需要高速时钟的片内外设提供服务，比如定时器和ADC采样等。当CPU休眠时，只要SMCLK开启，定时器和ADC仍可工作（一般待片内外设完成工作后触发中断，唤醒CPU去做后续工作）。
- 3) **ACLK: 辅助时钟 (Auxillary Clock)**，辅助时钟的频率很低，所以即使一直开启功耗也不大，当然关掉也是可以的。辅助时钟可以供给那些只需低频时钟的片内外设，比如LCD控制器，还可用于产生节拍时基，与定时器配合间歇唤醒CPU。

5.1 时钟系统

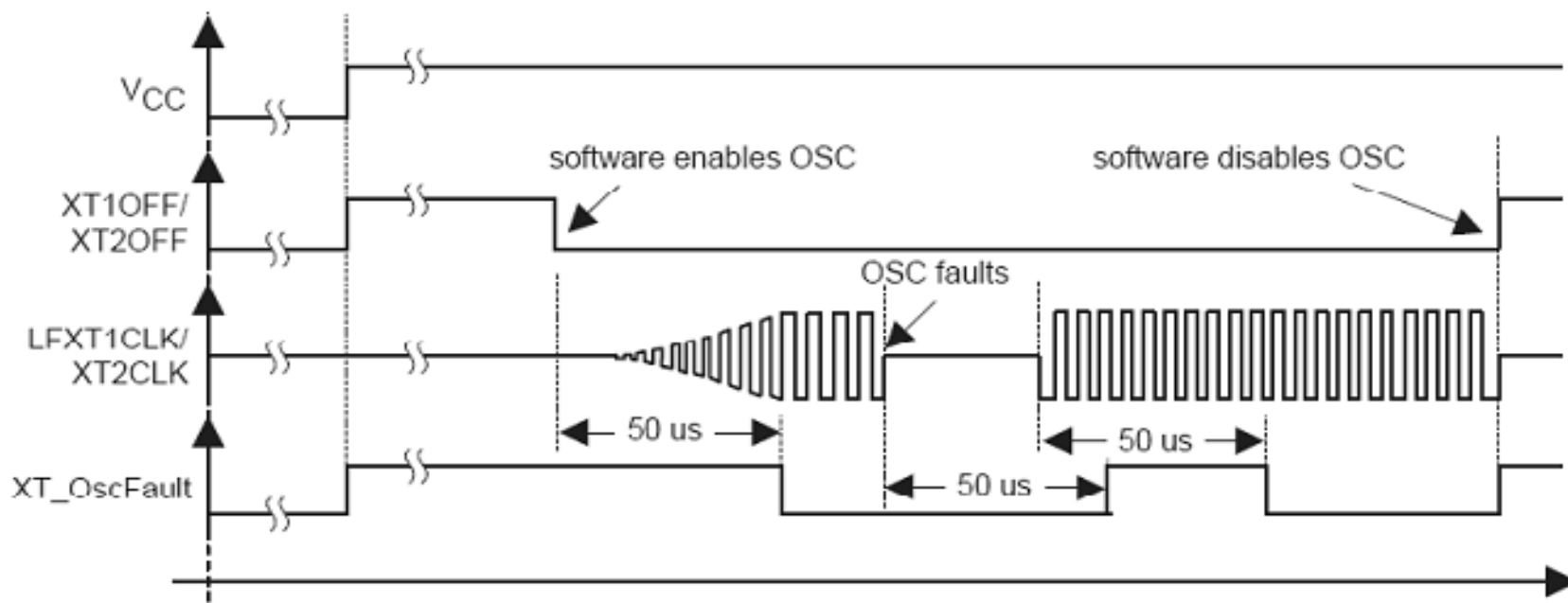


- 5.1.1 时钟系统结构
- 5.1.2 时钟失效处理



5.1.2 时钟失效处理

- **MSP430可检测多种时钟失效情况。例如，若XT2CLK失效50us，监测电路就能及时发现并使XT_OscFault信号复位。**





5.1.2 时钟失效处理

- 如果检测出振荡器失效，则置OFIFG=1。
- 发出非屏蔽中断申请，CPU执行完当前指令，响应该中断，转去执行相应中断程序。
- 如果此时MCLK来源于LFXT1(在HF模式)或XT2，DCO会自动被选作MCLK时钟源。
- 因此，在外部振荡器失效的情况下，程序可以继续执行。



5.1.2 时钟失效处理

● 中断标志寄存器1(IFG1)

7	6	5	4	3	2	1	0
						OFIFG	
						rw-1	

OFIFG: 用于振荡器失效中断标志位

OFIFG=0: 没有振荡器失效中断请求;

OFIFG=1: 有振荡器失效中断请求;



5.1.1 时钟系统结构

● 中断使能寄存器1(IE1)

7	6	5	4	3	2	1	0
						OFIE	
						rw-0	

OFIE: 用于振荡器失效中断使能位

OFIE=0: 振荡器失效中断关闭;

OFIE=1: 振荡器失效中断打开;